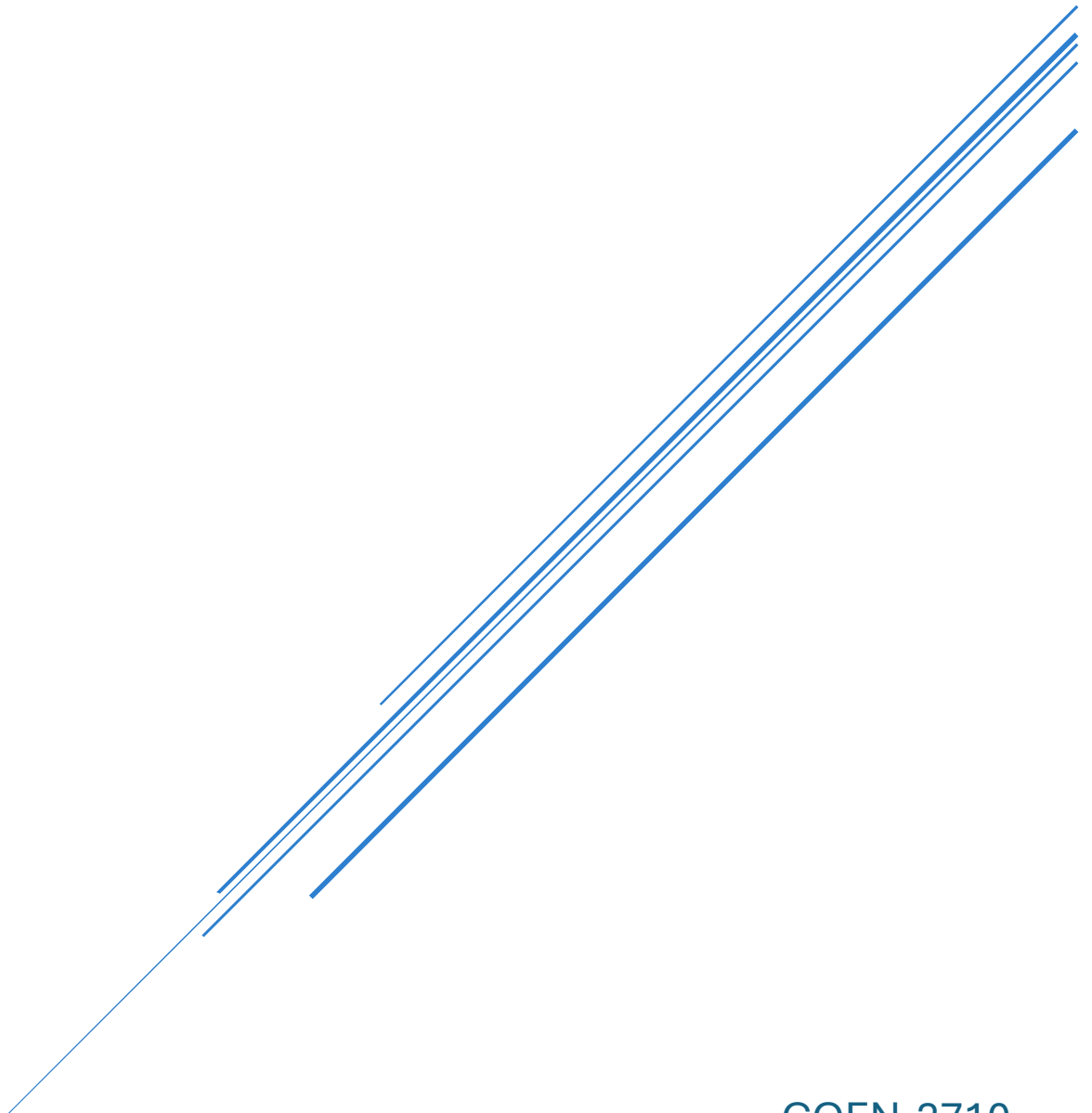


DESIGN PROJECT 2 PT.1

Microprocessors



COEN-2710

Annabel Gabriel, Tyler Giczkowski, Aidan VandenElzen

Summary:

For design project 2, part 1, the team was tasked to create the first 4 components found within the data path for the Risk-V operations ADD, SUB, AND, OR, LW, SW, and BEQ. These components were created in Vivado in the language VHDL. The components: 32-bit Adder, Register File, Main Control, and ALU Control have all been successfully tested utilizing test benches and waveform charts to monitor proper input-output functions. For each component, using Vivado's report timing summary, the longest timing delay can be determined. For the 32-bit Adder, the most critical path showed a timing delay of 14.089 ns. The Register File's most critical path showed a timing delay of 5.327 ns. The Main Control had a critical path with a timing delay of 5.924 ns. Lastly, the ALU Control's longest timing delay 4.887 ns.

Description:

32 Bit Adder:

The 32-bit adder was designed by refactoring the 4-bit Adder discussed in class adjusting both the input and output vectors to accommodate 32-bit addition. Additional carry signals were added to ensure all carry bits for each full adder was accounted for. The 32-bit Adder works by receiving two 32-bit words into aInput and bInput. The 32-bit Adder will then utilize 32 full adders to add $a(0)$ and $b(0)$ and carry $c(0)$ with outputs $s(0)$ for sum, and $c(1)$ for carry for the next full adder to use. All full adders were used as follows: for n bits, $FA(n)$: full_adder PORT MAP($x \Rightarrow a(n)$, $y \Rightarrow b(n)$, $cin \Rightarrow C(n)$, $sum \Rightarrow s(n)$, $cout \Rightarrow C(n+1)$). Observe figure 1 to find x , y , cin , sum , and $cout$ connections. Since this was a 32-bit Adder $n = 0$ to 31. $C32$ was mapped to "carry".

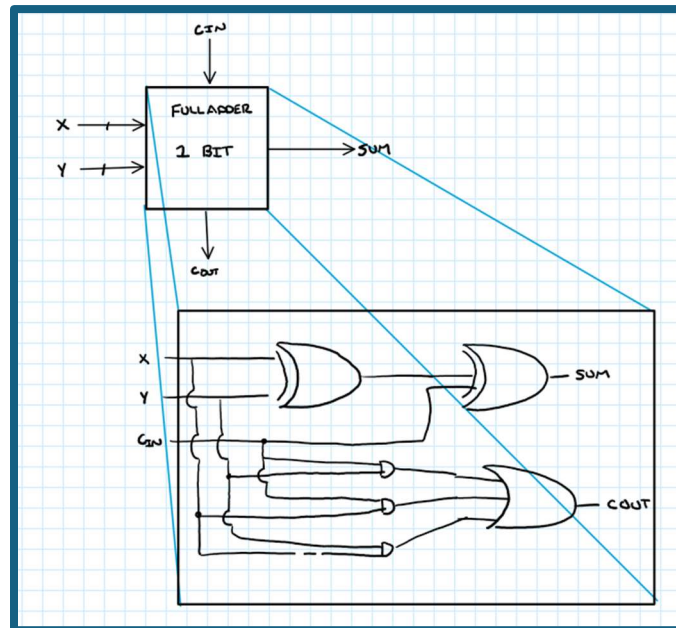


Figure 1 Full Adder Port Map and Logic Circuit

The 32-bit Adder will be utilized in the bigger picture of the processor as a method to add two words together within the ALU given the ALU Control signals for an add function. In which case the result of the adder will be on the output bus of the ALU for use by other components. Figure 2 shows the component diagram for the 32-bit Adder with two 32-bit bus inputs and one 32-bit bus output which holds the resulting arithmetic.

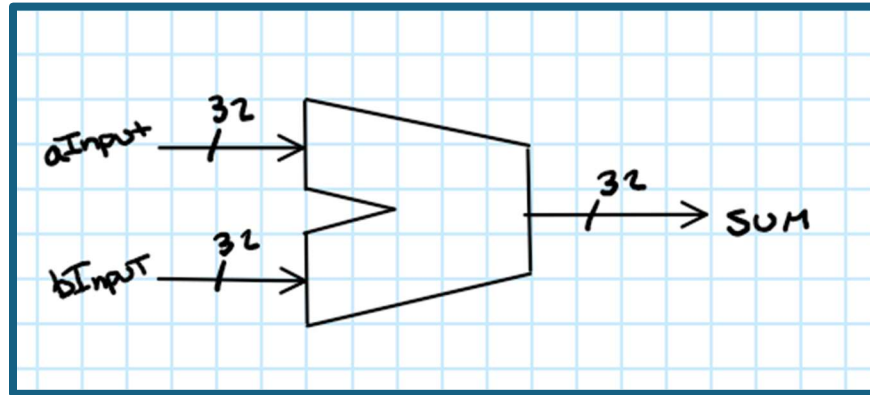


Figure 2 32-bit Adder Component Diagram

Figure 3 is the synthesized diagram for the designed 32-bit Adder provided by the Vivado tools. The diagram demonstrates the architecture of the final implementation of the adder as inputs aInput and bInput are on the far left, and the sum and carry out on the far right, connected by the 4-bit and full Adders.

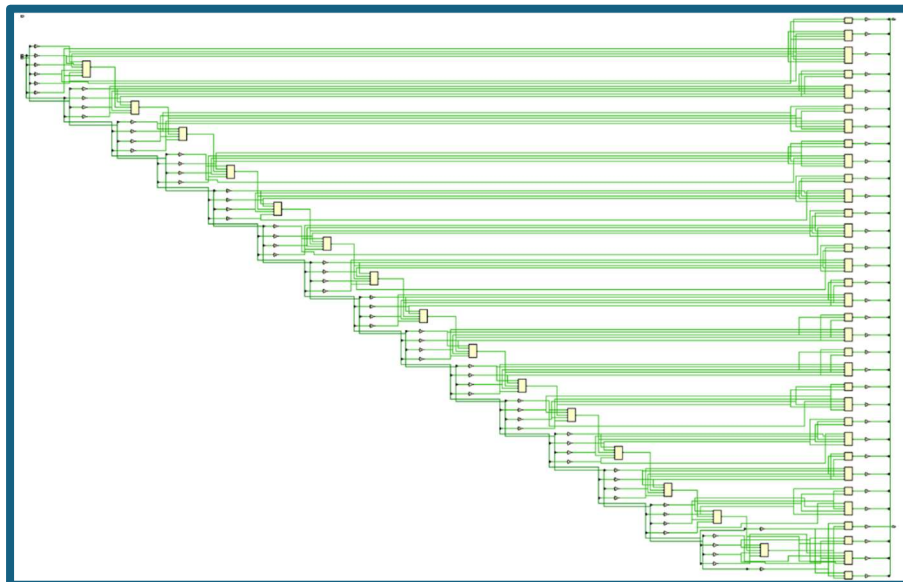


Figure 3 Synthesized Diagram for 32-bit Adder

Register File Component:

The Registers File Component has the inputs read_reg1, read_reg2, write_reg, and write_data. It utilizes the clock (clk) and write_en parts and outputs the data_out1 and data_out2. The read_reg1 and read_reg2 are asynchronous inputs. This means they are in isolation from the clock. Both inputs read the register input and output the data inside the corresponding register. This means that the input to read_reg1 will be the register number and the data_out1 will be whatever is read from that register. The write_reg and write_data depend on both the clock and write_en values. The write_reg will write a register when the write_en is set to 1 and the clock is on a rising edge. The write_data will write data into the corresponding register in the write_reg when the write_en is set to 1 and the clock is on a rising edge. The Registers File Component is used to read the initial instructions to execute the task needed. The Registers File schematic

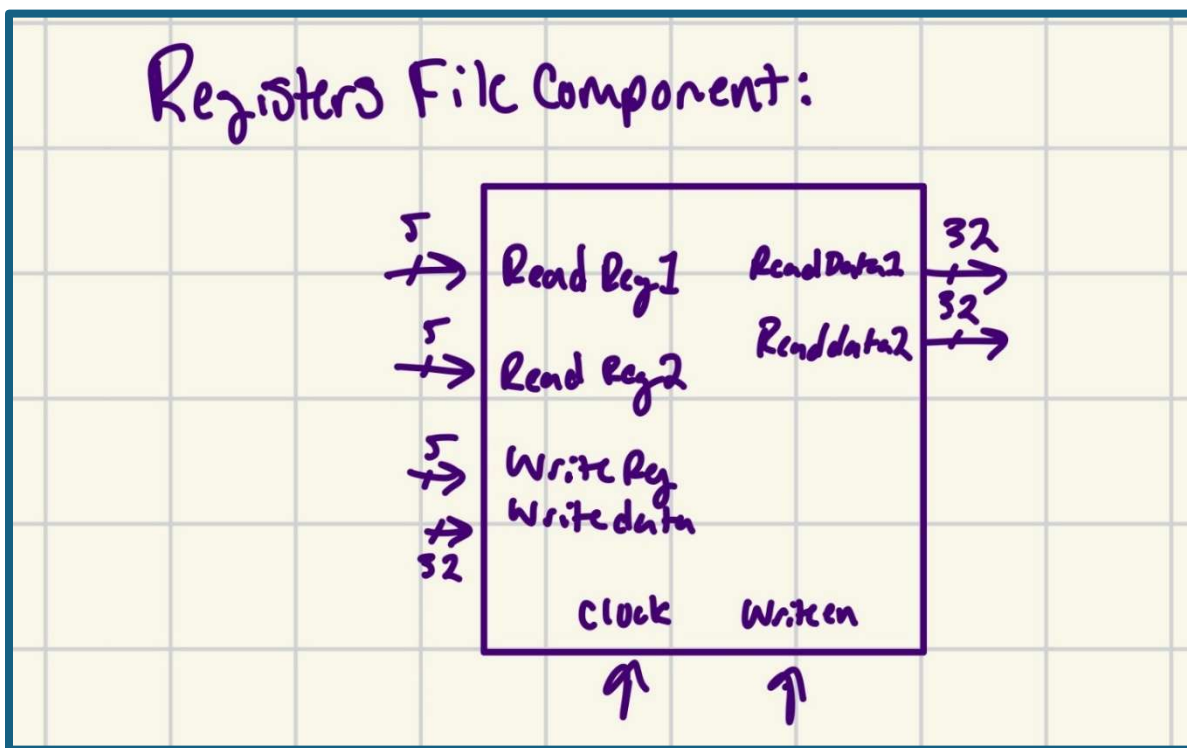


Figure 4 Register File Component Diagram

depicted in figure 5, depicts the analysis of the synthesized circuit. The full Registers File component is depicted in figure 4.

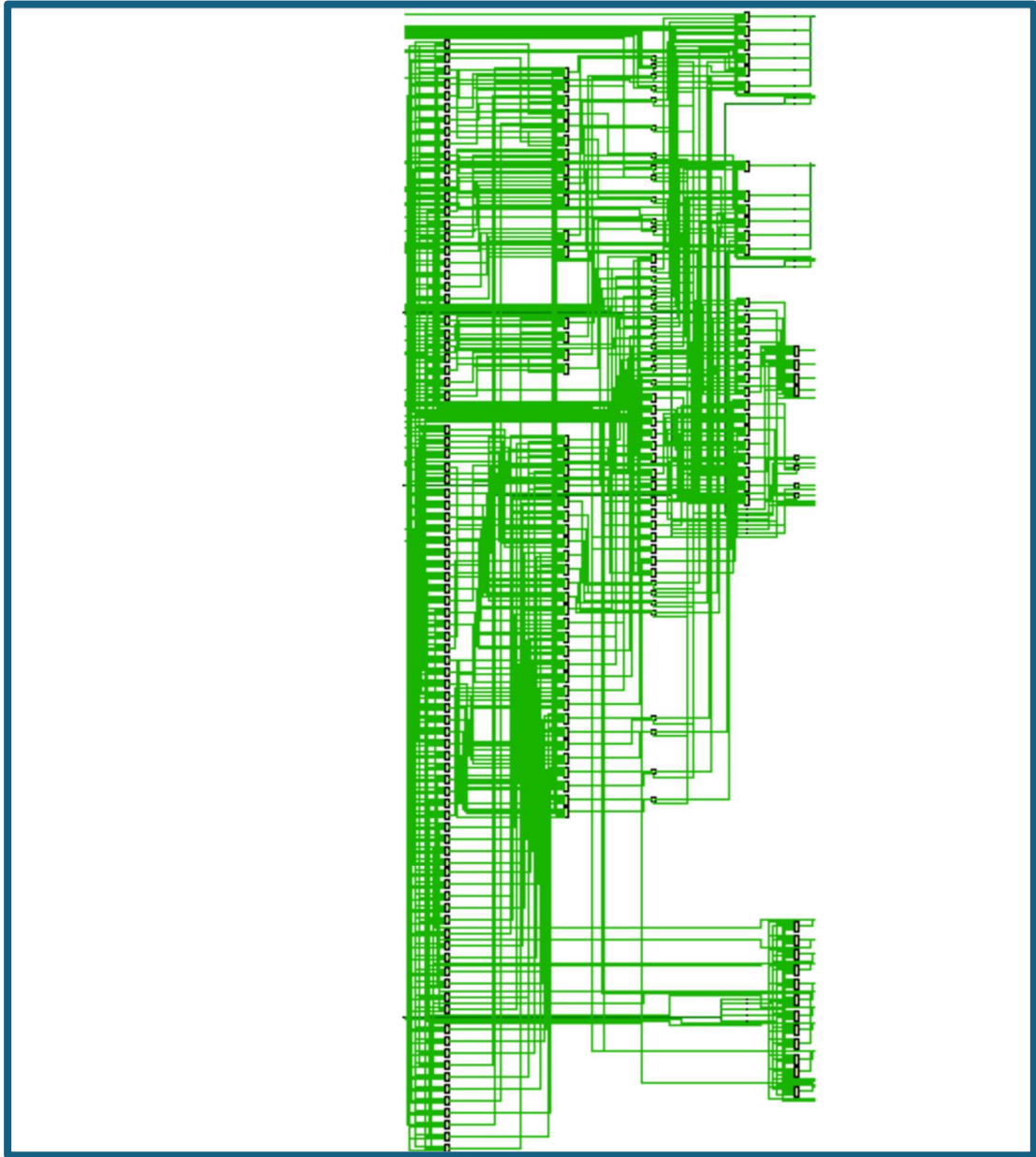


Figure 5 Schematic Diagram of the Register File Component

Main Control Component:

Table 1: Main Control Truth Table

Signal Name	R-Format	lw	sw	beq
I[6]	0	0	0	1
I[5]	1	0	1	1
I[4]	1	0	0	0
I[3]	0	0	0	0
I[2]	0	0	0	0
I[1]	1	1	1	1
I[0]	1	1	1	1
ALUSrc	0	1	1	0
MemtoReg	0	1	X	X
RegWrite	1	1	0	0
MemRead	0	1	0	0
MemWrite	0	0	1	0
Branch	0	0	0	1
ALUOp1	1	0	0	0
ALUOP0	0	0	0	1

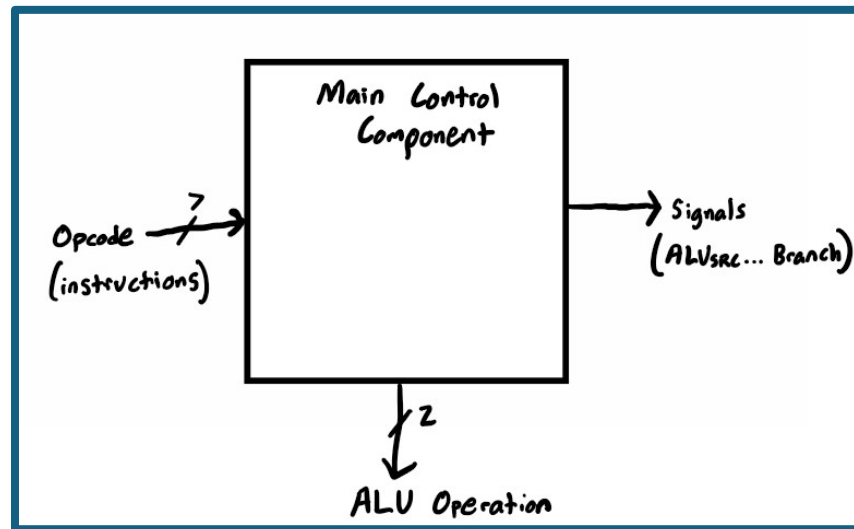


Figure 6 Black Box Diagram for Main Control Component

The Main Control Component was designed using the Boolean functions from the truth table in table 1 as well as the black box diagram referenced in Figure 6. The Main Control Component interprets the Opcode, $I[6]-I[0]$, to generate control signals that dictate instruction execution. It determines whether to read or write registers, corresponding to the RegWrite and MemtoReg instructions. To access memory from the MemRead and MemWrite instructions. Or to perform arithmetic using the ALUOp, or Branch. For R-Format instructions, the Opcode being 0110011, it enables register writing, where RegWrite = 1, and the ALUOp = 10. Load Word instructions, having an Opcode 0000011, sets MemRead = 1 and MemtoReg = 1 for memory access. Store Word instructions, with an Opcode of 010001, activates MemWrite = 1 for data storage. Branch Equal instructions, that have an Opcode of 1100011, sets Branch = 1 for conditional jumps. The schematic for this design is seen in Figure 7 below. The Main Control Unit, in the “big picture” of the project, will decode the Opcode, $I[6]-I[0]$, and generate control signals for different instruction types. It will direct data flow and ensure each instruction is executed correctly by enabling or disabling register writing, memory access, ALU operations, and branching.

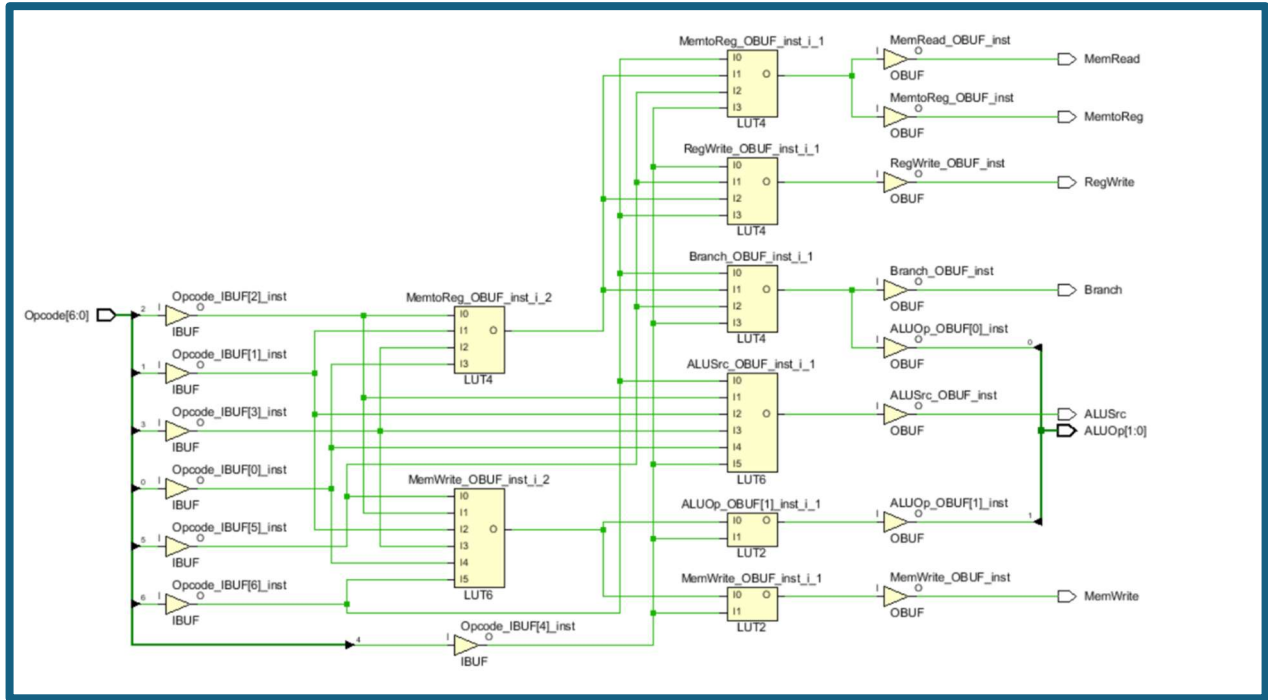


Figure 7 Synthesized Diagram for Main Control Component

ALU Control Component:

Table 2: ALU Operations and their corresponding binary outputs

ALU control lines	Function
0000	AND
0001	OR
0010	ADD
0110	SUB

Table 3: ALU Control Truth Table

ALUOp		Funct7 field							Funct3 Field			Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	0	1	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

The ALU Control component is based off of the truth table represented in table 3. This truth table describes the various inputs and outputs from the ALU Control. Depending on the various inputs of Funct3 (I_12 through I_14), Funct7 (I_25 through I_31), and ALUOp (ALU_Op1 and ALU_Op2) a different ALU operation is performed. Each ALU binary operation has a corresponding function each of which is described in table 2. ADD is 2. SUB is 6. AND is 0. OR is 1. The ALU Control Component is used to tell the ALU what operation to perform: ADD, SUB, AND, OR. The schematic design pictured in figure 7 shows the full data path for the ALU Control Component. The ALU Control Component is depicted in Figure 9.

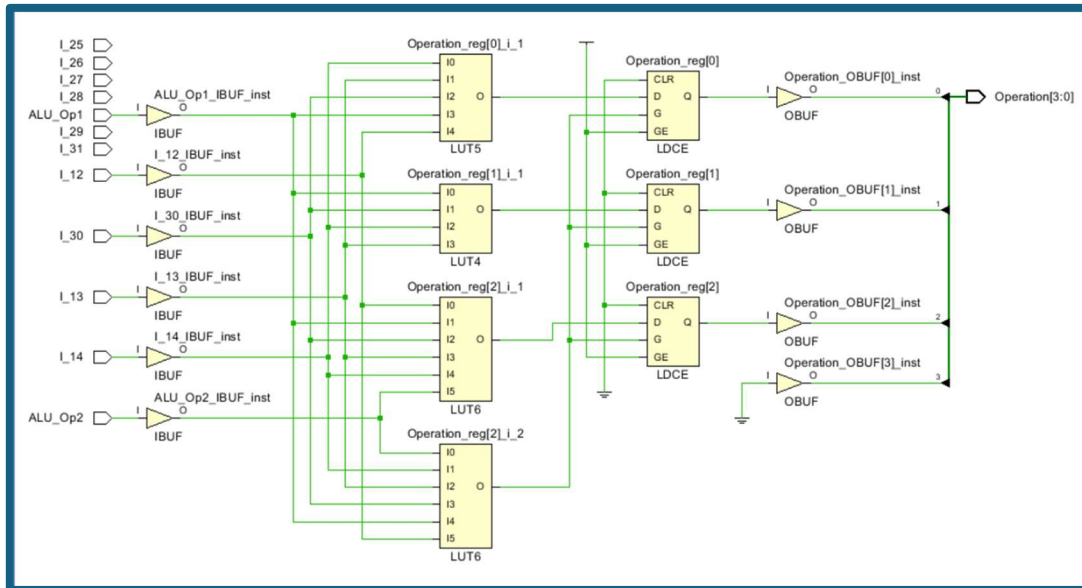


Figure 8 Synthesized Diagram for ALU Control

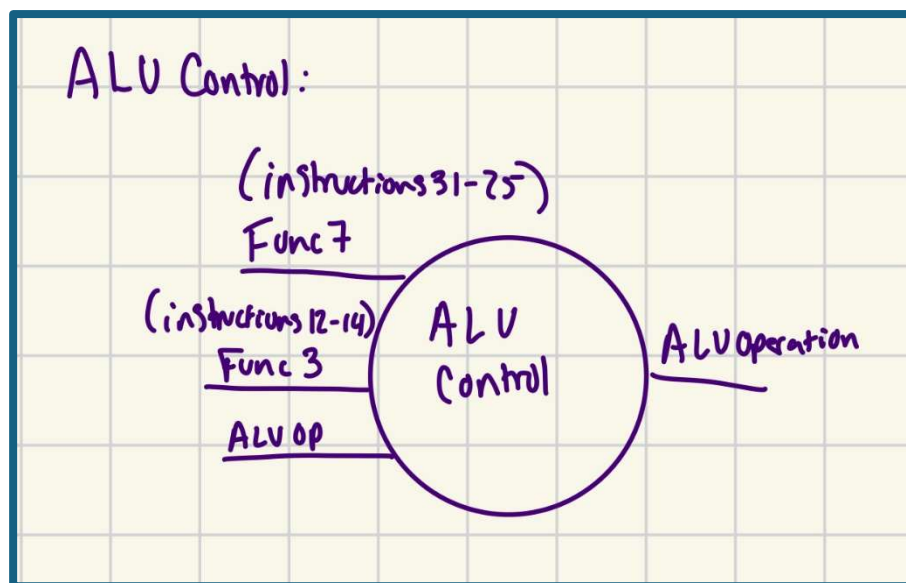


Figure 9 The Schematic Diagram of the Synthesized Circuit

Testing and Verification with Results:

32 bit-Adder Component:

To test the 32-bit Adder a test bench was used which input values into aInput, bInput, and carryIn to observe the output results “carry” and “summation.” The first test set aInput to 0x00000000, bInput to 0x00000000, and carryIn to 0 and 0 ns. The expected and produced results were summation = 0x00000000, and carry = 0. For the second test, at 10 ns, aInput = 0x00000001, and bInput = 0x00000001, the equivalent of 1 + 1. Therefore, the expected results were summation = 0x00000002, and carry = 0, which is what was produced. The next test was to determine if the carry output functioned as expected. At 20 ns, aInput = 0xffffffff, and bInput = 0x00000001 which is the equivalent of 1 + (-1). The adder is designed to show a carry of 1, and the sum of a number that overflows will result in summation = 0x00000000, which is correct for $1 + (-1) = 0$. The wave forms show that this occurs, as carry flips to 1, and summation = 0x00000000 at 20 ns. The next test performs a similar case where at 30 ns aInput = 0x80000000 and bInput = 0x80000000. These inputs should result in a overflow and the carry would continue to be 1. The last test is a simple test of proper addition that uses a number that needs 6 bits to properly represent. At 40 ns, aInput = 0x0000001f and bInput = 0x0000001f (31 + 31). The expected result is a carry of ‘0’, and a summation of 0x0000003e, which is the proper result. At 50 ns all inputs are set to 0 to observe proper a reset and all outputs return to 0 as expected. The tests described was in figure 10 (below).

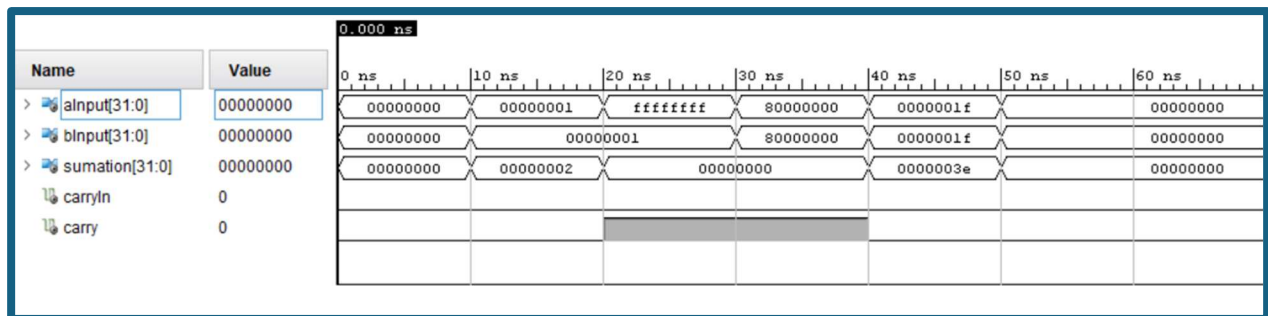


Figure 10 32-bit Adder Simulation Waveform (Test)

Register File Component:

The Register File Component was tested in the testbench depicted in Figure 11. The first test that was done was setting the clock period. Throughout the entire testbench the clock has a period of 10 ns. The next test is done to the reset function. At 20 ns, the reset function goes from 0 to 1. The next test is done to the write_reg and write_data. At 25 ns the write enable is set to 1 and the clock is on a rising edge making it, so the write functions are able to happen. At that same time, the write register is set to 5 and the write data is set to AAAAAAAA. This sets the data in register 5 to be AAAAAAAA. 10 ns later, the write register is set to 10 and the write data is set to 55555555. This sets the data in register 10 to be 55555555. The next test is done to the read functions. At 55 ns the input in the register read 1 is set to 5. This reads the data in register 5

and outputs it into data_out1. In data_out1, AAAAAAAA is output which is the same input put into register 5 previously. The same thing is done for the second read register. During this the register 10 is read and the output 55555555 is output. The final test is done to the write enable. To test this the write enable is set to 0 so data cannot be written. The write functions attempt to write 12345678 into register 11. However, when it is read at 75 ns, the register's data reads as 0. This makes sense because this is the data it was initialized to be.

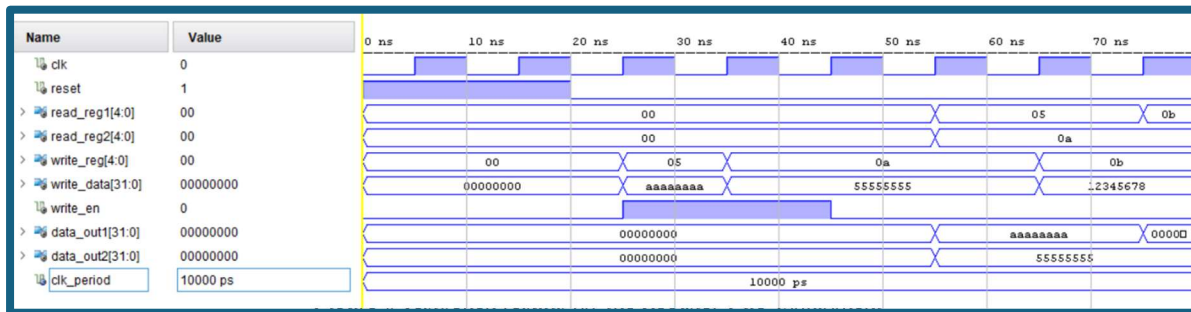


Figure 11 Register File Simulation Waveform (Test)

ALU Control Component:

The ALU Control Component was tested in the testbench depicted in Figure 12. For 6 different inputs, the output of the ALU Control was analyzed. At 0 ns, the ALU_Op1 and ALU_Op2 were both set to 0. Based off the Boolean functions placed into the driver, the Operation done is equivalent to 0010. When looking at the truth table depicted in Table X, this remains true. The operation 0010 is equivalent to the operation ADD so when these inputs are true the operation ADD is done. The next test changes the ALU_Op1 to 1. This also outputs 0010. That is the second possible function that could produce the ADD operation. At 20 ns, the I_30 turns on. This changes the output operation to 0110 which is the ALU operation SUB. At 30 ns, the ALU_Op2 switches to 1 and the output operation remains 0110. This is the second function possible to produce the SUB function. At 40 ns, I_12, I_13, and I_14 all turn to 1. This switches the output operation to be 0000 which is the function AND. At 50 ns, the I_12 turns to 0. This switches the output operation to be 0001 which is the ALU operation OR. All of these values align with Table 2 and Table 3.

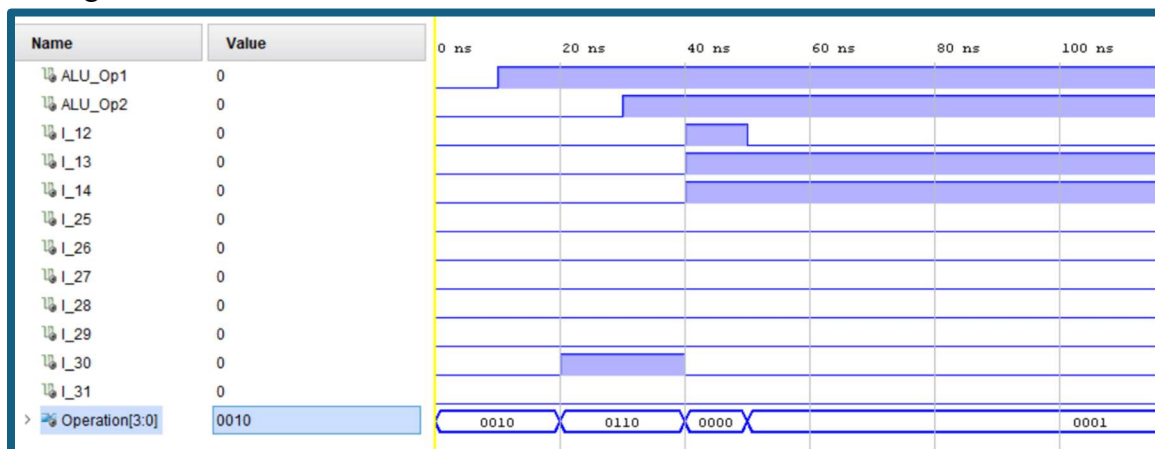


Figure 12 Testbench results for the ALU Control Component

Main Control Component:

The main control component was tested using a testbench depicted in Figure 13. The inputs and outputs for the main control component were noted from the truth table in Table 1. The component was tested using a base case. At 0ns, the Opcode was set to 0000000. Since this does not correspond to any valid instruction in the truth table, all control signals remained at 0, indicating an idle state.

At 50ns, the Opcode changed to 0110011, which corresponds to an R-Format instruction. As expected, the control signals were set where ALUSrc = 0, MemtoReg = 0, RegWrite = 1, MemRead = 0, MemWrite = 0, Branch = 0, and the ALUOp = 10, indicative of an R-Type ALU operation.

At 100ns, the Opcode was set to 0000011, corresponding to a load word instruction. The outputs changed to ALUSrc = 1, MemtoReg = 1, RegWrite = 1, MemRead = 1, MemWrite = 0, Branch = 0, and the ALUOp = 00, indicative of an ADD ALU operation.

At 150ns, the Opcode was set to 0100011, used for a store word instruction. The outputs changed to ALUSrc = 1, MemtoReg = 0(X), RegWrite = 0, MemRead = 0, MemWrite = 1, Branch = 0, and the ALUOp = 00, indicative of an address calculation (using addition) ALU operation.

At 200ns, the Opcode was set to 1100011, corresponding to a branch on equal instruction. The outputs expectedly changed as the ALUSrc = 0, MemtoReg = 0(X), RegWrite = 0, MemRead = 0, MemWrite = 0, Branch = 1, and the ALUOp = 01, indicative of a comparison (using subtraction) ALU operation.

Throughout the simulation, the Main Control Component correctly generated control signals according to the truth table, ensuring the processor's correct behavior based on instruction opcodes. The observed results validate that the Main Control Unit is functioning as expected.

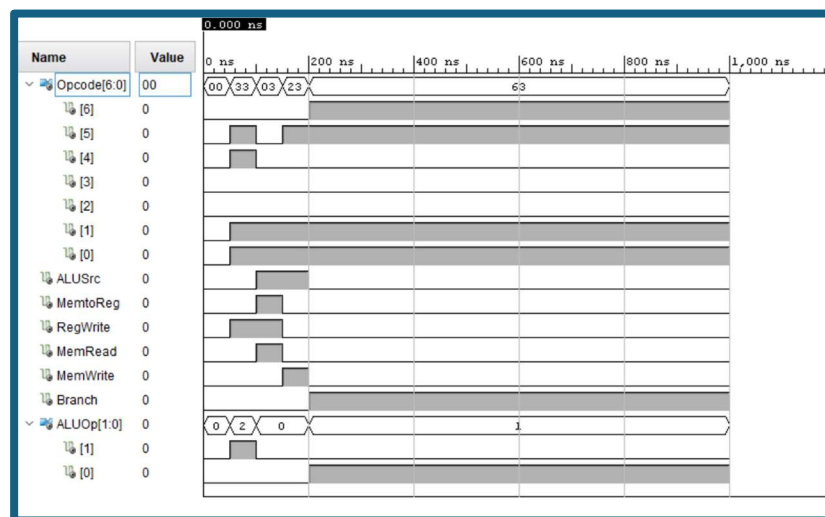


Figure 13 Test Bench for Main Control Component

Performance Analysis:

32-bit Adder Component:

The timing analysis for the 32 Bit Adder revealed the longest timing delay for the circuit was from the input aInput, to the output carryFlag. The total delay of this path was 14.089 ns. Figure 14 demonstrates this by showing only the most-critical path that this delay time references, starting at aInput, and ending at carryFlag.

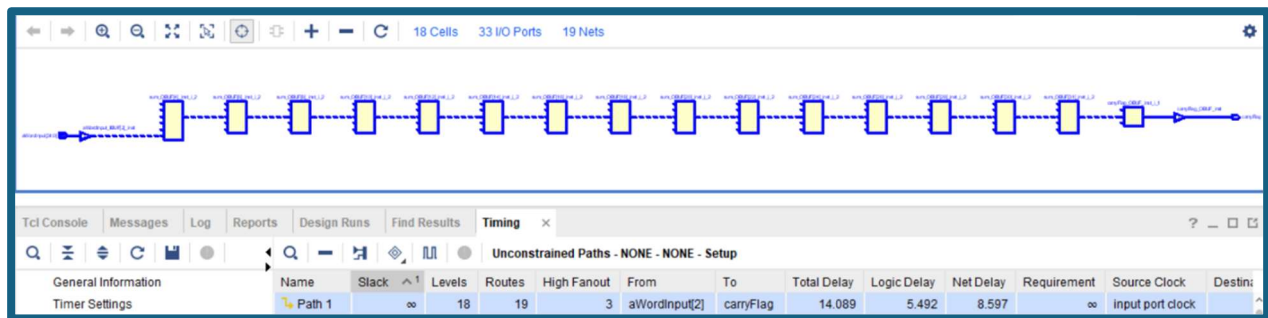


Figure 14 Most-critical Path for 32-bit Adder

ALU Control Component:

The timing analysis for the ALU control component the longest timing delay for the circuit was from the input I_12 to the output Operation. The total delay of this path was 4.887 ns. Figure 15 demonstrates this by showing only the most-critical path that this delay time references, starting at I_12, and ending at Operation.

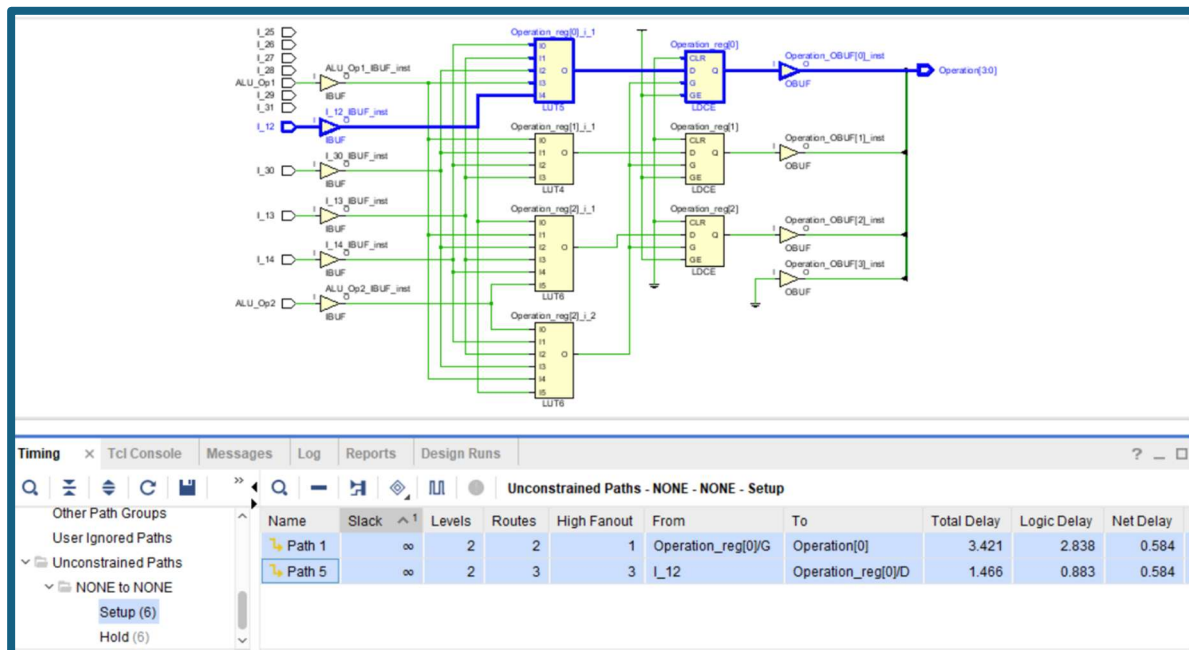


Figure 15 Most-critical Path for ALU Control Component

Register File Component:

The timing analysis for the Register File Component reveals that the longest timing delay for the circuit was from the input `read_reg1` to the output `data_out1`. The total delay of this path was 5.327 ns. Figure 16 demonstrates this by showing only the most-critical path that this delay time references, starting at `read_reg1`, and ending at `data_out1`.

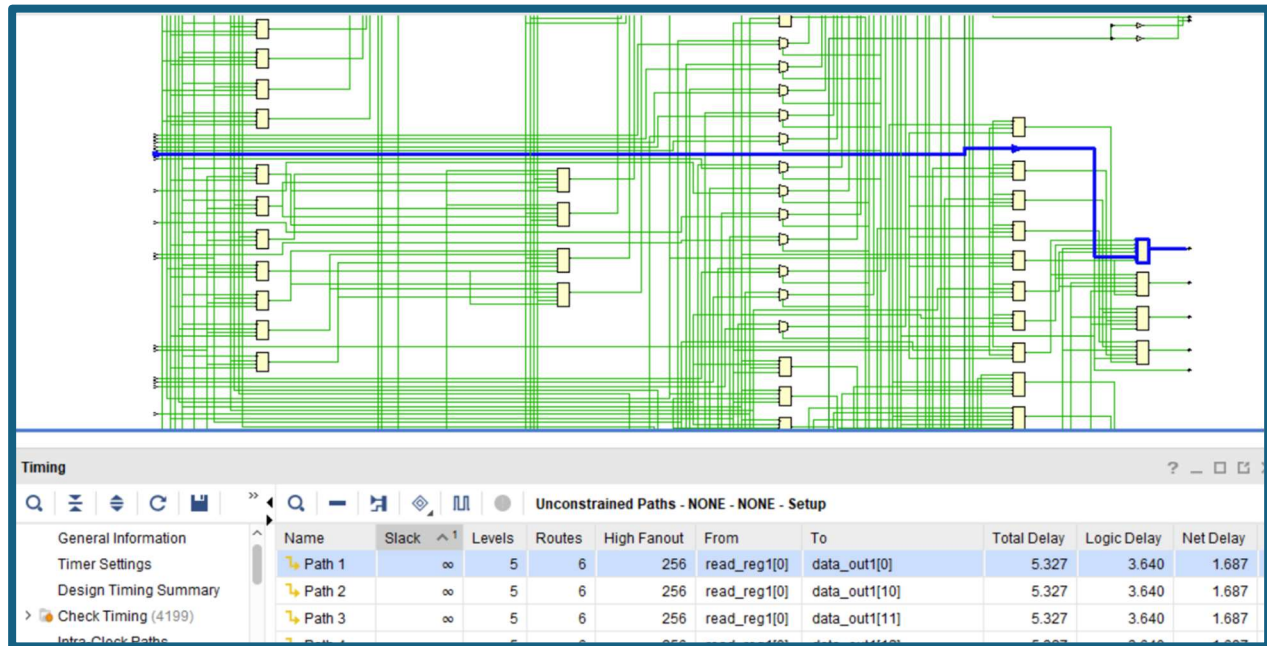


Figure 16 Most-critical Path for Register File

Main Control Component:

The timing analysis for the Main Control Component articulates that the longest timing delay for the circuit was from the Opcode input in which the MemtoReg instructions were executed before the branch instructions, and finally to the ALUOp signals. The total delay of this path was 5.924ns. Figure 17 demonstrates this by showing only the most critical path that this delay time references, starting with the Opcode inputs and ending at the ALUOp signal outputs.

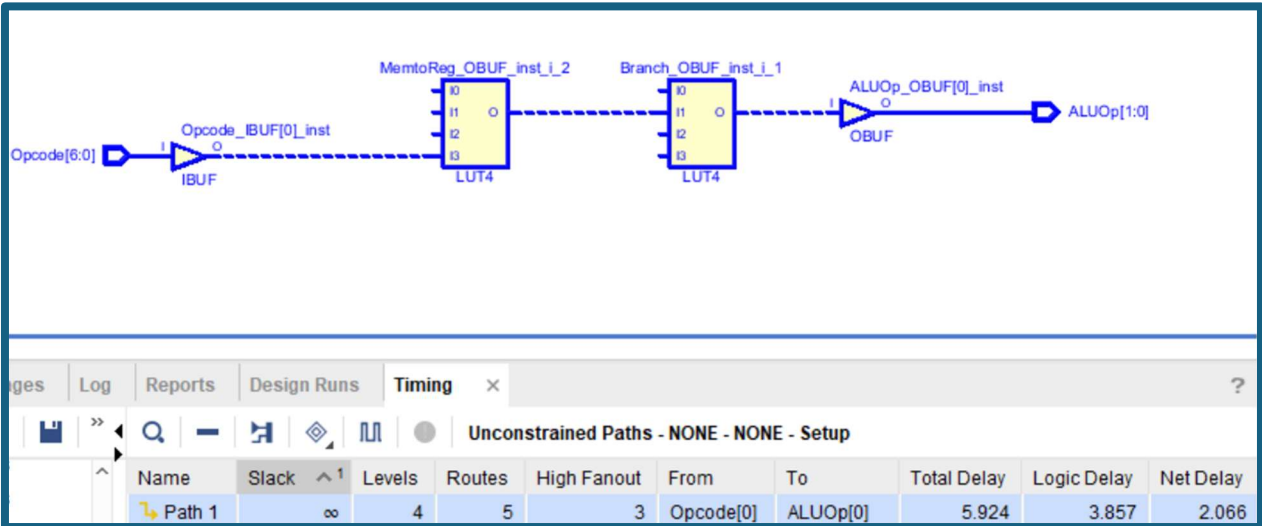


Figure 17 Most-critical Path for Main Control

Conclusions:

The purpose of this project was to design and implement the main components of the RISC-V processor. To do this, entity declarations and descriptions were made in VHDL to describe components such as, a 32-bit adder, register file, main control, and ALU control. The team used *Xilinx's Vivado WebPack* to design and simulate the components using VHDL description language [1]. These descriptions were made from analyzing Boolean logic from truth tables provided by *Computer Organization and Design RISC-V Edition: The Hardware Software Interface* [2].

Each member of the group worked to translate the truth table into an entity description based on the component that they were implementing. This format is referenced and executed in the textbook *DIGITAL DESIGN USING DIGILENT FPGA BOARDS: Vhdl* (Haskell, R. 2017) [3]. This taught to not only examine the truth tables but to understand and develop digital logic to test these functions in simulations. With this came new understanding of running simulations on VHDL software and performance analysis.

Challenges came when attempting to meet and work on the project together as each member has busy schedules. However, most of the work completed during this project was done independently and then discussed and observed when the team had the opportunity to be together and work on the project. Other challenges included hidden syntax errors or incorrect analysis of the Boolean functions that through off testing and measurements. Paired programming was very useful in these scenarios to find either the syntax error or incorrect input.

References:

- [1] *Installer Information*. (n.d.). AMD. <https://www.xilinx.com/support/installer/installer-info-general.html>
- [2] Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann.
- [3] Haskell, R. (2017). *DIGITAL DESIGN USING DIGILENT FPGA BOARDS: Vhdl*.